

FORMATION EMBARQUÉE

Antisèche — C Embarque

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

Antisèche — C embarqué

Types (toujours explicites)

```
#include <stdint.h>
uint8_t u8;      // 0..255           int8_t s8;      // -128..127
uint16_t u16;    // 0..65535         int16_t s16;    // -32768..32767
uint32_t u32;    // 0..4 294 967 295 int32_t s32;
bool b;         // <stdbool.h>      size_t taille;
```

⚠ `int` = 16 bits sur AVR/Arduino, 32 bits sur ARM → jamais de `int` nu pour une valeur qui compte.
Pas de `float` sans FPU : **point fixe** (stocker des centièmes en `int32_t`).

Les 5 opérations de bits (à savoir de tête)

```
r |= (1u << n);      // SET      mettre le bit n à 1
r &= ~(1u << n);     // CLEAR   mettre le bit n à 0
r ^= (1u << n);      // TOGGLE  inverser le bit n
if (r & (1u << n)) {} // TEST    le bit n est-il à 1 ?
champ = (r >> pos) & msk; // EXTRAIRE un champ (décaler PUIS masquer)

r = (r & ~(0x3u << 2)) | (val << 2); // écrire un champ sans toucher au reste
```

Masques utiles

`0x0F` bits 0-3

`0xF0` bits 4-7

`0xFF` un octet

`0xFFFF` deux octets

`x & 0xFF` garder l'octet bas

`(x >> 8) & 0xFF` prendre l'octet haut

`(hi << 8) | lo` assembler 16 bits

`1u << n` bit n seul

Pointeurs

```
uint8_t x = 42;
uint8_t *p = &x; // p contient l'ADRESSE de x
*p = 100;        // écrit DANS x
p[2] == *(p + 2); // arithmétique : avance par taille d'élément

const uint8_t *p; // données non modifiables via p
uint8_t *const p; // adresse figée
volatile uint32_t *reg; // registre matériel (le cas standard)

#define GPIOC_ODR (*(volatile uint32_t *)0x4001100C) // accès registre
```

Pièges mortels : pointeur non initialisé · retour d'adresse d'une variable locale · `strcpy` sans borne (utiliser `snprintf`).

| volatile — et ses limites

Obligatoire pour : registres matériels · variables partagées avec une ISR · variables partagées entre tâches RTOS.

```
volatile uint8_t drapeau; // écrit par l'ISR, lu par main
```

⚠ **volatile** ≠ **atomique**. Un `uint32_t` sur CPU 8 bits se lit en 4 fois → section critique :

```
noInterrupts(); copie = compteur; interrupts(); // Arduino
__disable_irq(); copie = compteur; __enable_irq(); // ARM
```

| Machine d'états (le patron n°1)

```
typedef enum { REPOS, ACTIF, ERREUR } Etat;
static Etat etat = REPOS;
static uint32_t t_entree;

void tick(uint32_t maintenant) {
    switch (etat) {
        case REPOS:
            if (demarrer) { etat = ACTIF; t_entree = maintenant; }
            break;
        case ACTIF:
            if (maintenant - t_entree >= DUREE) { etat = REPOS; ... }
            break; // ↑ soustraction : robuste au wrap
        case ERREUR: securiser(); break;
    }
}
```

| Temps non bloquant

```
if (maintenant - dernier >= PERIODE) { dernier = maintenant; /* action */ }
```

✓ `maintenant - dernier >= P` ✗ `maintenant >= dernier + P` (casse au débordement à 49,7 jours pour un `uint32_t` en ms).

| Structures & trames

```
typedef struct { uint8_t id; int16_t val; } Mesure;
Mesure m = { .id = 1, .val = 253 };
Mesure *pm = &m; pm->id = 2;
```

⚠ **Padding** : `sizeof(struct)` ≥ somme des champs. Ne JAMAIS caster un buffer réseau en struct → décoder champ par champ :

```
val = (int16_t)((((uint16_t)buf[2] << 8) | buf[3])); // big-endian
```

Squelette bare-metal & règles d'or

```
int main(void) {  
    init_materiel();  
    while (1) { /* lire → décider → agir, jamais bloquant */ }  
}
```

1. Compiler `-Wall -Wextra -Werror`, zéro warning.
2. Pas de `malloc` après l'init (fragmentation, échec imprévisible).
3. ISR : courte, un drapeau, pas de `printf` / `delay` / allocation.
4. Vérifier chaque retour de fonction d'E/S.
5. Tester les bords : 0, max, débordement, négatif, entrée invalide.

Compilation

```
gcc -Wall -Wextra -O2 main.c -o app # PC  
avr-gcc -mmcu=atmega328p -DF_CPU=16000000UL -Os blink.c -o blink.elf  
arm-none-eabi-gcc -mcpu=cortex-m4 -mthumb -Os main.c  
gcc -S main.c # voir l'assembleur godbolt.org en ligne
```